PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC YEAR: 2026–27

DEEP LEARNING COURSE PROJECT REPORT

DEEPFAKE AUDIO DETECTION USING HYBRID CNN-LSTM NEURAL NETWORKS



Faculty Name: Dr. G. Lalitha Kumari

Team Members:

1. **K. Veerendra Babu 23501A0592**
2. **Md. Sadhika 23501A05A9**
3. **M. Navya 23501A05B4**

ABSTRACT

The rapid development of generative audio systems has made it possible to create speech that closely resembles recordings of real people. Although synthetic speech has useful applications, it can also be misused for impersonation, misinformation, and voice-based fraud. This project investigates a Deep Learning approach for distinguishing genuine audio from audio labelled as spoofed or synthetic in the selected dataset.

The proposed system uses a hybrid Convolutional Neural Network and Long Short-Term Memory network (CNN-LSTM). Audio recordings are loaded and standardized, then represented as log-Mel spectrograms. These spectrograms describe how acoustic energy changes across frequency bands over time. CNN layers learn local time-frequency patterns, while the LSTM component models sequential information across spectrogram frames. A final classification layer produces a score for the spoof class.

The project uses the DFADD dataset resource associated with research on diffusion- and flow-matching-based audio deepfakes. Dataset labels, split names, and file structure must be verified against the version used in the notebook. Model performance is evaluated on held-out data using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix. An optional Gradio interface allows a user to upload an audio file and view the model prediction.

The system is an educational research prototype rather than a forensic authentication service. Its performance depends on dataset coverage, label quality, train/test separation, preprocessing choices, and the similarity between evaluation audio and real-world recordings. Final metric values must be inserted from the completed notebook run; this report does not assume unverified results.

KEYWORDS: Audio Deepfake Detection, Synthetic Speech, DFADD, Log-Mel Spectrogram, CNN, LSTM, Deep Learning, Binary Classification.

INTRODUCTION

Speech is used in personal communication, education, entertainment, customer service, and identity verification. Recent advances in generative models have improved the quality of artificial speech and voice conversion. In some cases, synthetic recordings can sound natural enough that listeners may find it difficult to identify them by hearing alone.

Audio deepfakes can be created for legitimate purposes such as accessibility, dubbing, and creative production. However, the same capabilities may be abused to imitate a person, spread misleading recordings, or support voice-based social engineering. These risks motivate research into automated methods that analyse audio signals for patterns associated with generated or manipulated speech.

Traditional approaches to audio spoofing detection may use hand-crafted acoustic features, signal-processing measures, or statistical classifiers. Deep Learning provides an alternative by learning representations from training examples. In this project, log-Mel spectrograms are used as the input representation for a hybrid CNN-LSTM model.

The CNN processes local regions of the spectrogram and learns patterns across time and frequency. The LSTM receives a sequence of learned features and models relationships across frames. The output layer estimates whether an input resembles the spoof class represented in the training data. This prediction is a statistical model output, not definitive proof of how an audio recording was created.

1.1 Background

The audio signal contains information about pitch, timbre, spectral shape, timing, and other characteristics. Synthetic speech systems may reproduce many natural characteristics but can also leave detectable patterns. A machine-learning detector attempts to learn discriminative patterns from labelled examples. The reliability of such a detector must be measured on audio that was not used to fit the model.

1.2 Motivation

The motivation for this project is to demonstrate how Deep Learning can be applied to a current multimedia security problem. The project combines audio preprocessing, feature extraction, neural-network design, model evaluation, and a simple prediction interface in one end-to-end workflow.

PROBLEM STATEMENT

Synthetic and manipulated audio can resemble genuine speech, making manual identification unreliable in some circumstances. The characteristics of generated speech also change as synthesis systems improve. An automated system is therefore useful for investigating whether acoustic features learned from a labelled dataset can distinguish genuine recordings from spoofed or synthetic examples.

This project addresses the problem by developing a CNN-LSTM classifier that accepts an audio recording, converts it to a log-Mel spectrogram, and predicts a dataset-defined class. The task is limited to the labels available in the selected dataset and does not identify the speaker, determine intent, or establish authenticity with certainty.

Objectives of the Project

1. To study audio deepfake detection as a binary classification problem.
2. To load and inspect a labelled audio dataset and verify its class definitions.
3. To standardize audio recordings using a consistent sample rate, channel format, and duration.
4. To extract log-Mel spectrograms as neural-network input features.
5. To design a hybrid CNN-LSTM architecture for learning time-frequency and sequential patterns.
6. To train the model using training data and monitor learning with validation data.
7. To evaluate the final model on a separate test set using multiple metrics.
8. To visualize model performance with a confusion matrix and ROC curve.
9. To provide an audio prediction function and, if enabled in the notebook, a Gradio upload interface.
10. To document limitations and identify improvements for future experimentation.

LITERATURE REVIEW

Research on audio spoofing detection includes replay-attack detection, voice-conversion detection, text-to-speech spoofing, and newer forms of AI-generated speech. Public evaluation campaigns such as ASVspoof have helped researchers compare countermeasures under defined protocols. Their findings also highlight the importance of evaluating systems on data that reflects different attack types and recording conditions.

Signal-processing and handcrafted-feature approaches

Earlier systems often used acoustic descriptors and statistical classifiers. Such methods can be computationally efficient and interpretable, but their effectiveness depends on the chosen features and whether those features capture differences present in the target data.

Deep-learning approaches

Neural networks can learn representations directly from waveforms or from time-frequency features. Convolutional models are suitable for spectrogram inputs because they can learn local patterns across neighbouring time and frequency regions. Recurrent networks, including LSTM networks, can model sequential dependencies and preserve useful information over a sequence.

Hybrid CNN-LSTM approach

A hybrid CNN-LSTM combines two complementary operations: convolutional layers extract local representations, and recurrent layers process a sequence of those representations. In this project, the hybrid architecture is used to analyse log-Mel spectrograms. Its usefulness must be determined experimentally; the architecture alone does not guarantee superior performance.

Dataset relevance

The DFADD dataset is associated with research into audio deepfakes generated by diffusion and flow-matching methods. The original paper and dataset card should be consulted to confirm dataset composition, labels, access conditions, and the appropriate evaluation protocol. This report does not claim a comparative result against published systems because no such experiment is included in the project notebook.

The literature discussion is a project-focused summary. In a final academic submission, any specific claim drawn from a paper should be cited directly to that paper, and the dataset version used in the experiment should be documented.

EXISTING SYSTEM

Existing audio deepfake detection methods include signal-processing analysis, handcrafted acoustic features, conventional machine-learning classifiers, and end-to-end deep neural networks. Some systems analyse raw waveforms, while others use representations such as spectrograms or cepstral features. Their performance can vary depending on the generator, language, recording environment, compression, and dataset used for evaluation.

Common limitations of existing approaches

- Known-pattern dependence: a detector may perform poorly when tested on synthesis methods that are absent from its training data.
- Domain mismatch: microphones, codecs, background noise, and speaking styles can change the acoustic distribution.
- Dataset bias: a model may learn dataset-specific artifacts rather than general indicators of synthetic speech.
- False alarms and misses: genuine audio can be flagged as spoof, and spoofed audio can be classified as genuine.
- Evaluation sensitivity: results can be misleading if near-duplicate clips or related speakers occur in both training and test sets.
- Limited explainability: a predicted score does not necessarily reveal which acoustic cues influenced the model.

These limitations motivate a carefully evaluated model that learns from spectrogram features. The proposed project is not assumed to eliminate the limitations; it provides a practical framework for testing a hybrid CNN-LSTM model under a documented dataset and split.

PROPOSED SYSTEM

The proposed system classifies audio recordings using a hybrid CNN-LSTM neural network. It begins by loading the dataset and checking the available labels and audio fields. Each usable recording is converted to a consistent waveform representation. The audio is resampled, converted to mono when required, and cropped or padded to the configured duration.

The preprocessed waveform is transformed into a log-Mel spectrogram. The spectrogram is passed through convolutional layers that learn local time-frequency patterns. Pooling operations reduce the feature-map size, and the resulting representation is arranged as a sequence for the LSTM layer. Dense layers and a sigmoid output produce the model's spoof score.

Advantages of the proposed approach

- Combines local acoustic feature extraction with temporal sequence modelling.
- Uses a spectrogram representation that can be inspected and visualized.
- Provides a repeatable preprocessing and inference pipeline.
- Uses held-out evaluation data to estimate performance beyond the training set.
- Reports multiple metrics rather than relying on accuracy alone.
- Can save the trained model for later inference when the model and preprocessing settings are retained.
- Can be demonstrated through an audio upload interface if the interface cell is run successfully.

System output

For each accepted recording, the system returns a score interpreted according to the class mapping used during training. The user interface should describe the output as a model prediction and should not present it as conclusive evidence that a recording is authentic or fake.

SYSTEM REQUIREMENTS

Hardware requirements

- Computer or laptop with a modern processor.
- Sufficient RAM for loading audio examples and creating spectrogram features.
- Internet access to obtain the dataset and install libraries.
- GPU acceleration is recommended for faster neural-network training when available; a compatible CPU runtime may be used for smaller experiments.

Software requirements

- Python programming language.
- Google Colab or a compatible Jupyter notebook environment.
- TensorFlow/Keras for model construction and training.
- Librosa for audio loading and feature extraction.
- NumPy and Pandas for numerical operations and metadata handling.
- Scikit-learn for data splitting and evaluation metrics.
- Matplotlib and Seaborn for plots and confusion-matrix visualization.
- Hugging Face Datasets or the dataset's supported download method, depending on the version used.
- Gradio for the optional interactive interface.

Category	Role in this project
Python	Controls data processing, training, evaluation, and inference.
TensorFlow/Keras	Defines, trains, saves, and loads the CNN-LSTM model.
Librosa	Loads waveforms and computes audio features.
Scikit-learn	Supports splitting and classification metrics.
Google Colab	Provides notebook execution and optional GPU resources.
Gradio	Provides an upload-and-predict interface when enabled.

DATASET DESCRIPTION

The project is designed around the DFADD dataset resource associated with the study "The Diffusion and Flow-Matching Based Audio Deepfake Dataset." The resource focuses on audio deepfake detection and is relevant to synthetic speech produced by newer generative approaches. The precise subset used in this project must be recorded from the dataset configuration and the notebook output.

Before training, the dataset loader should be inspected to identify the audio column, label field, split names, and label meanings. Labels must be mapped only after confirming their definitions in the dataset documentation. The genuine/spoofed names used in plots and the interface must correspond to that mapping.

Dataset checks

- Record the dataset name, repository or source, version, and access date.
- Inspect the dataset features and sample records before preprocessing.
- Count examples in each class and each split.
- Check for missing audio, unreadable files, and ambiguous labels.
- Preserve official train/validation/test partitions when the benchmark provides them.
- Avoid leakage from duplicate recordings or closely related samples across partitions.
- Confirm licensing and any restrictions on redistribution of audio files.

Dataset summary table

Field	Value to verify from the actual notebook
Dataset	DFADD; confirm exact configuration and version
Task	Dataset-defined genuine versus spoof/synthetic classification
Audio input	Audio waveform or dataset audio field
Class mapping	Use the official label meanings; document mapping here
Training/validation/test counts	[Insert actual counts after loading]
Preprocessing exclusions	[Record skipped or unreadable examples]

The final report should use the exact counts printed by the notebook. The dataset's total size should not be guessed from an online description because versions and subsets can differ.

METHODOLOGY AND SYSTEM FLOW

The system follows a sequence of stages, beginning with labelled audio data and ending with a prediction and evaluation report. Each stage must use consistent settings so that training and inference inputs have the same shape and meaning.

11. Load dataset metadata and audio examples.
12. Verify labels and create the class mapping.
13. Load audio and convert it to the configured sample rate and channel format.
14. Crop or pad the waveform to the configured duration.
15. Compute a log-Mel spectrogram using the notebook's feature parameters.
16. Create training, validation, and test partitions without data leakage.
17. Construct and compile the CNN-LSTM network.
18. Train using training data and monitor validation performance.
19. Load the best saved checkpoint and evaluate it on test data.
20. Generate metrics and plots, then test an audio upload through the inference function.

Flow chart

Step	Operation	Output
1	Dataset loading and label verification	Labelled audio samples
2	Audio preprocessing	Standardized waveforms
3	Log-Mel feature extraction	Spectrogram tensors
4	Data partitioning	Train / validation / test sets
5	CNN feature extraction	Learned time-frequency maps
6	LSTM sequence learning	Temporal representation
7	Dense + sigmoid classifier	Spoof-class score
8	Evaluation and inference	Metrics, plots, and prediction

Figure description: Dataset → Label verification → Audio preprocessing → Log-Mel spectrogram → CNN → LSTM → Sigmoid classification → Evaluation / prediction.

The flow above describes the proposed implementation. Any stage not successfully completed in the final notebook should be reported as planned work rather than as a completed feature.

DATA PREPROCESSING AND FEATURE EXTRACTION

Audio standardization

Audio files can differ in sample rate, number of channels, and duration. The preprocessing function converts each recording into the expected format. When the model is configured for mono audio, stereo channels are combined into a single channel. The waveform is resampled to the sample rate specified in the notebook, then cropped or padded to a fixed duration.

Log-Mel spectrogram

A spectrogram represents the distribution of signal energy over time and frequency. A short-time Fourier transform (STFT) analyses overlapping frames of the waveform. A Mel filter bank then groups spectral energy into frequency bands arranged on the Mel scale. Applying logarithmic compression reduces the dynamic range and creates a compact representation suitable for neural-network input.

$$L(m,t) = \log(M(m,t) + \varepsilon)$$

Here, $M(m,t)$ denotes the Mel-band energy at band m and frame t, and ε is a small positive constant that avoids taking the logarithm of zero. The number of Mel bands, FFT size, hop length, and normalization must be the same for training and inference.

Why use spectrogram features?

- They expose changes in acoustic energy across time and frequency.
- They allow convolutional layers to learn local structures in a two-dimensional representation.
- They provide a consistent fixed-shape input after duration normalization.
- They can be plotted to inspect examples and verify preprocessing.

Preprocessing consistency

The uploaded test recording must pass through the same loading, resampling, duration adjustment, and feature-extraction operations used during model training. If preprocessing differs, the input distribution and tensor shape may not match what the model learned.

DATASET SPLITTING AND CNN-LSTM ARCHITECTURE

Dataset splitting

Training data are used to update model weights. Validation data are used to monitor learning, select the best checkpoint, and identify possible overfitting. Test data are reserved for the final performance estimate and should not be used repeatedly to tune model choices.

If the dataset supplies official benchmark partitions, those partitions should be preserved where feasible. If a new split is created, the split method, random seed, and class counts must be documented. Where metadata permits, a speaker- or generator-independent test protocol is preferable because a random split may overestimate performance on unseen synthesis systems.

Partition	Purpose	Actual count
Training	Fit model parameters	[Insert from notebook]
Validation	Monitor learning and select checkpoint	[Insert from notebook]
Testing	Final evaluation on held-out samples	[Insert from notebook]

CNN-LSTM model architecture

The architecture combines convolutional feature extraction and recurrent sequence learning. The precise layer count and dimensions should match the model definition in the final notebook.

Component	Function
Input	Accepts a log-Mel spectrogram tensor.
Convolutional layers	Learn local patterns across time and frequency.
Activation and pooling	Introduce non-linearity and reduce feature-map dimensions.
Dropout / normalization	Help regularize training and stabilize learning when used.
Reshape / sequence conversion	Arranges CNN features as a sequence for the recurrent layer.
LSTM layer	Models dependencies across time frames.
Dense layer	Combines the learned representation for classification.
Sigmoid output	Returns a score for the class mapped as spoof.

The model output is a probability-like score produced by the sigmoid layer. It should be interpreted in the context of the dataset and threshold used; calibration is not guaranteed.

MODEL TRAINING AND IMPLEMENTATION DETAILS

Model training

The CNN-LSTM model is compiled with a binary classification loss and an optimizer selected in the notebook. During training, batches of spectrograms are passed through the network, predictions are compared with their labels, and gradients are used to update the parameters. Validation performance is monitored after each epoch.

Early stopping and model checkpointing can be used to prevent unnecessary training and retain the best validation checkpoint. A learning-rate reduction callback may also be used when validation performance stops improving. The final report should record the actual optimizer, loss, batch size, epoch limit, random seed, and callbacks from the executed code.

Implementation modules

21. Environment setup: imports libraries and installs required packages.
22. Dataset loading: accesses the selected DFADD configuration and inspects its fields.
23. Label preparation: verifies the class meanings and maps them to numeric labels.
24. Audio preprocessing: standardizes waveform format and duration.
25. Feature extraction: generates log-Mel spectrograms.
26. Data partitioning: separates training, validation, and test samples.
27. Model construction: defines the CNN-LSTM architecture.
28. Model training: fits the network and saves the selected checkpoint.
29. Evaluation: computes classification metrics and produces plots.
30. Inference: preprocesses a new audio file and returns the model score.
31. Interface: optionally exposes inference through a Gradio upload page.

Model saving

For reproducible inference, save the trained model together with the preprocessing configuration, sample rate, duration, Mel-feature parameters, class mapping, and dataset notes. A model file alone is not sufficient if the preprocessing code or label mapping is lost.

ALGORITHM

Input: A labelled audio dataset with genuine and spoof/synthetic class labels.

Output: A trained CNN-LSTM model, evaluation metrics, and an audio classification function.

Algorithm steps

32. Start the program and import the required libraries.
33. Load the chosen DFADD dataset configuration.
34. Inspect the dataset fields, split names, and label meanings.
35. Remove or record samples that are missing, unreadable, or ambiguously labelled.
36. Map the verified labels to the project's binary classes.
37. Load each audio recording and standardize its channel format and sample rate.
38. Crop or pad each waveform to the configured duration.
39. Compute a log-Mel spectrogram for each recording.
40. Create or preserve training, validation, and test partitions.
41. Construct the CNN-LSTM neural network.
42. Compile the model with the selected optimizer and binary loss.
43. Train on the training partition while monitoring validation performance.
44. Save the best model checkpoint.
45. Generate predictions for the held-out test partition.
46. Calculate accuracy, precision, recall, F1-score, and ROC-AUC.
47. Generate the confusion matrix and ROC curve.
48. Load a user-provided audio file and apply the same preprocessing.
49. Return the predicted class and score with an appropriate caution.
50. Save the model, metrics, plots, and configuration; stop.

Inference logic

For an input waveform, the inference function performs preprocessing, creates the log-Mel tensor, adds the batch and channel dimensions required by the model, and obtains a prediction score. A threshold is then applied to map the score to one of the two class labels. The threshold and class mapping must be consistent with the evaluation code.

PERFORMANCE EVALUATION METRICS

Performance is measured on held-out test data. A single metric cannot describe all errors, so the project uses complementary measures.

Metric	Meaning in this project
Accuracy	Fraction of all test recordings classified correctly.
Precision (spoof)	Of the recordings predicted as spoof, the fraction labelled spoof.
Recall (spoof)	Of all spoof-labelled test recordings, the fraction detected.
F1-score	Harmonic mean of precision and recall.
ROC-AUC	Measures how well scores rank positive examples above negative examples across thresholds.
Confusion matrix	Counts true and incorrect predictions for each class.
Loss curves	Show training and validation loss over epochs and can reveal overfitting.

Confusion-matrix terms

- True Positive (TP): a spoof recording correctly predicted as spoof.
- True Negative (TN): a genuine recording correctly predicted as genuine.
- False Positive (FP): a genuine recording incorrectly predicted as spoof.
- False Negative (FN): a spoof recording incorrectly predicted as genuine.

Interpretation

High recall can be useful when missing a spoof recording is costly, but low precision may lead to many genuine recordings being flagged. High accuracy can also be misleading if the classes are imbalanced. The confusion matrix, class-wise precision and recall, ROC-AUC, and class distribution should therefore be considered together.

For a fair estimate of generalization, the test set should remain separate from model fitting and hyperparameter decisions. If recordings from the same speakers or generators occur across partitions, this should be acknowledged as a limitation.

RESULTS AND DISCUSSION

The completed notebook should report the measured performance of the trained CNN-LSTM model on the selected test partition. The evaluation process produces predicted scores, converts them to class predictions using the configured threshold, and calculates the metrics listed in the previous section. The notebook may also produce a confusion matrix, ROC curve, and training-history plots.

No numerical results are inserted here because a verified output from the completed training run was not supplied while preparing this report. Do not report estimated or expected values as experimental findings. After execution, copy the actual class counts and metric values from the notebook output into the table below.

Evaluation item	Observed result from notebook
Dataset version / configuration	[Fill in exact source and configuration]
Training examples	[Insert actual count]
Validation examples	[Insert actual count]
Test examples	[Insert actual count]
Test accuracy	[Insert measured value]
Spoof precision	[Insert measured value]
Spoof recall	[Insert measured value]
Spoof F1-score	[Insert measured value]
ROC-AUC	[Insert measured value]
Best epoch / checkpoint	[Insert actual value if logged]

Discussion points to complete

- Describe whether training and validation curves suggest underfitting or overfitting.
- Explain the most common errors visible in the confusion matrix.
- Compare precision and recall and discuss the impact of false positives and false negatives.
- State the test split protocol and any overlap risks involving speakers or synthesis systems.
- Mention runtime, compute limitations, and any preprocessing or dataset issues encountered.

The final conclusion should be based on these observed results. Avoid claims that the model detects every deepfake or generalizes to all unseen audio-generation systems unless those claims have been tested.

APPLICATIONS AND ADVANTAGES

Potential applications

51. Media verification research: a model score may help prioritize audio for further examination.
52. Voice-security studies: supports experiments on synthetic speech and spoofing countermeasures.
53. Digital literacy demonstrations: illustrates why audio alone may not always establish authenticity.
54. Academic research: provides a practical example of audio preprocessing and hybrid neural-network modelling.
55. Content moderation workflows: could be investigated as one signal in a broader review process.
56. Synthetic-speech benchmarking: can support controlled comparisons when evaluation protocols and datasets are documented.

These are potential uses, not capabilities automatically provided by the notebook. The project does not independently verify a speaker's identity, trace the creator of an audio clip, or replace human review.

Advantages

- Uses log-Mel spectrograms to represent acoustic information across time and frequency.
- Combines convolutional pattern learning with LSTM sequence modelling.
- Supports quantitative evaluation on held-out data.
- Produces multiple metrics and visualizations for analysis.
- Can be reused for inference when the trained model and preprocessing configuration are saved.
- Can expose predictions through a simple upload interface if the optional Gradio component is executed successfully.

LIMITATIONS AND FUTURE ENHANCEMENTS

Limitations

- The model may misclassify genuine recordings or fail to identify spoofed audio.
- Performance depends on the dataset's labels, diversity, and coverage of synthesis methods.
- A random split may overestimate performance if related speakers, recording sessions, or generators occur in both train and test sets.
- Audio compression, background noise, language, microphone differences, and post-processing can affect predictions.
- A model trained on one dataset may not generalize to newer generation systems.
- The output score may not be calibrated as a real-world probability of deepfake status.
- The system does not identify the speaker or prove the origin of a recording.
- The interface, if publicly shared, should not be used to upload confidential or sensitive audio without appropriate safeguards.

Future enhancements

57. Evaluate with generator-independent and speaker-independent test partitions.
58. Compare the hybrid model with CNN-only, LSTM-only, and established audio-classification baselines.
59. Test robustness to noise, compression, re-recording, and different sample rates.
60. Explore self-supervised speech representations where compute resources permit.
61. Calibrate decision thresholds and report error trade-offs for different use cases.
62. Add clearer handling for unsupported, very short, or corrupted audio files.
63. Document dataset provenance, version, license, and preprocessing configuration.
64. Improve the interface with explanations, model version information, and a clear warning that outputs are not definitive.

CONCLUSION

This project investigates Deep Learning for audio deepfake detection using a hybrid CNN-LSTM architecture. The workflow standardizes audio recordings, extracts log-Mel spectrogram features, learns local time-frequency patterns with convolutional layers, and models temporal relationships with an LSTM layer. A classification head produces a score for the spoof class defined by the dataset.

The project also includes an evaluation plan based on accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix, together with a prediction function and optional upload interface. The actual model performance must be taken from the completed notebook run and interpreted in light of the dataset split, class balance, and possible overlap between related recordings.

The work demonstrates an end-to-end audio classification workflow and provides a foundation for further experiments. It should be regarded as a research prototype, not as a definitive method for authenticating audio. Stronger conclusions would require testing on diverse, recent, and independently sourced recordings, with careful separation of speakers and synthesis systems.

REFERENCES

[1] J. Du et al., “The Diffusion and Flow-Matching Based Audio Deepfake Dataset,” arXiv preprint arXiv:2409.08731, 2024. Available: https://arxiv.org/abs/2409.08731

[2] DFADD dataset resource, Hugging Face. Available: https://huggingface.co/datasets/isjwdu/DFADD. Confirm the exact dataset configuration and current terms before use.

[3] DFADD project repository, GitHub. Available: https://github.com/isjwdu/DFADD

[4] TensorFlow, “Keras: The high-level API for TensorFlow.” Available: https://www.tensorflow.org/guide/keras

[5] Librosa documentation. Available: https://librosa.org/doc/latest/index.html

[6] ASVspoof, official audio spoofing and countermeasure challenge resources. Available: https://www.asvspoof.org/

Final verification before submission

- Verify that the dataset configuration and label mapping match the executed notebook.
- Replace all result placeholders with measured outputs.
- Ensure the described CNN-LSTM layers match the actual model definition.
- Add any required screenshots of the actual output and interface.

- Follow college citation rules and check similarity using the institution's approved process.